Debug and look it up

A routine for getting unstuck

1 Read the error

The last line usually names the error and points toward the problem.

2 Check the shape

```
Look for missing colons, parentheses, quotes, indentation, or
misspelled names.
```

3 Trace values

```
Use print(variable) to see what a value is during the program.
```

4 Test a small piece

Run one line, one condition, or one function call before testing the whole program.

Good search

Search the exact goal: Python add item to list or Python range start stop step.

Final check

Can you explain what each variable stores, when each loop stops, and what each function returns?

Python Field Guide

A beginner lookup guide

Use this book while you code

```
Look up a pattern, copy its shape, then change the names and values
for your program.
```

You do not need to memorize everything

Programmers look up syntax. Your job is to understand what the code is doing and test it.

Concepts inside

Variables Lists Conditions For loops While loops Functions return None

How to read Python

Trace one line at a time

Assignment

`coins = 3` means store 3 in the name coins. The = sign assigns a value.

Indentation

Indented lines belong to the if, loop, or function above them.

Function call

`print(coins)` runs the function named print with coins as its argument.

Predict then run

Before you press Run, predict the output. Then compare your prediction with Python.

Functions return and None

Give reusable code a name

Define and call

```
def power(base, bonus):
return base + bonus

score = power(7, 5)
```

Vocabulary

Parameters are names in the definition. Arguments are values sent into the call.

return

return sends a value back to the caller. Here, power(7, 5) returns 12.

None

None means no value. A function without return still runs, but its result is None.

Print is different

print shows text on screen. return gives a value back so the program can reuse it.

While loops

Repeat while a condition stays True

Basic pattern

```
lives = 3
while lives > 0:
lives = lives - 1
```

Ask two questions

What condition keeps the loop running? What changes so the loop can stop?

Avoid infinite loops

If the condition never becomes False, the loop never stops. Update the condition variable inside the loop.

Choose the loop

```
Use for for a list or known count. Use while when repetition depends
on a condition.
```


Variables and lists

Store one value or a collection

Variables

```
name = "Luigi"
lives = 3
is_ready = True
```

Lists

```
items = ["key", "star"]
items.append("mushroom")
```

List lookup

items[0] is the first item. Python starts counting indexes at 0.

Useful checks

len(items) counts items. value in items checks whether a value is present.

Conditions and logic

Choose which code runs

Basic pattern

```
if coins >= 10:
print("1-UP")
else:
print("Keep going")
```

Comparisons

== equal != not equal > greater < less >= at least <= at most

Logic

and needs both conditions. or needs either condition. not reverses True and False.

Remember

`Use == to compare. Use = to assign a value.`

For loops and range

Repeat a known number of times

Loop through a list

```
for enemy in ["Boo", "Goomba"]:
print(enemy)
```

What range means

`range(4) produces 0, 1, 2, 3. It starts at 0 and stops before 4.`

Three useful forms

```
range(4) -> 0,1,2,3
range(2, 5) -> 2,3,4
range(2, 9, 2) -> 2,4,6,8
```

Accumulator

```
total = 0
for coin in [2, 4, 6]:
total = total + coin
```